

SPLAY TREES

1

BASIC IDEA

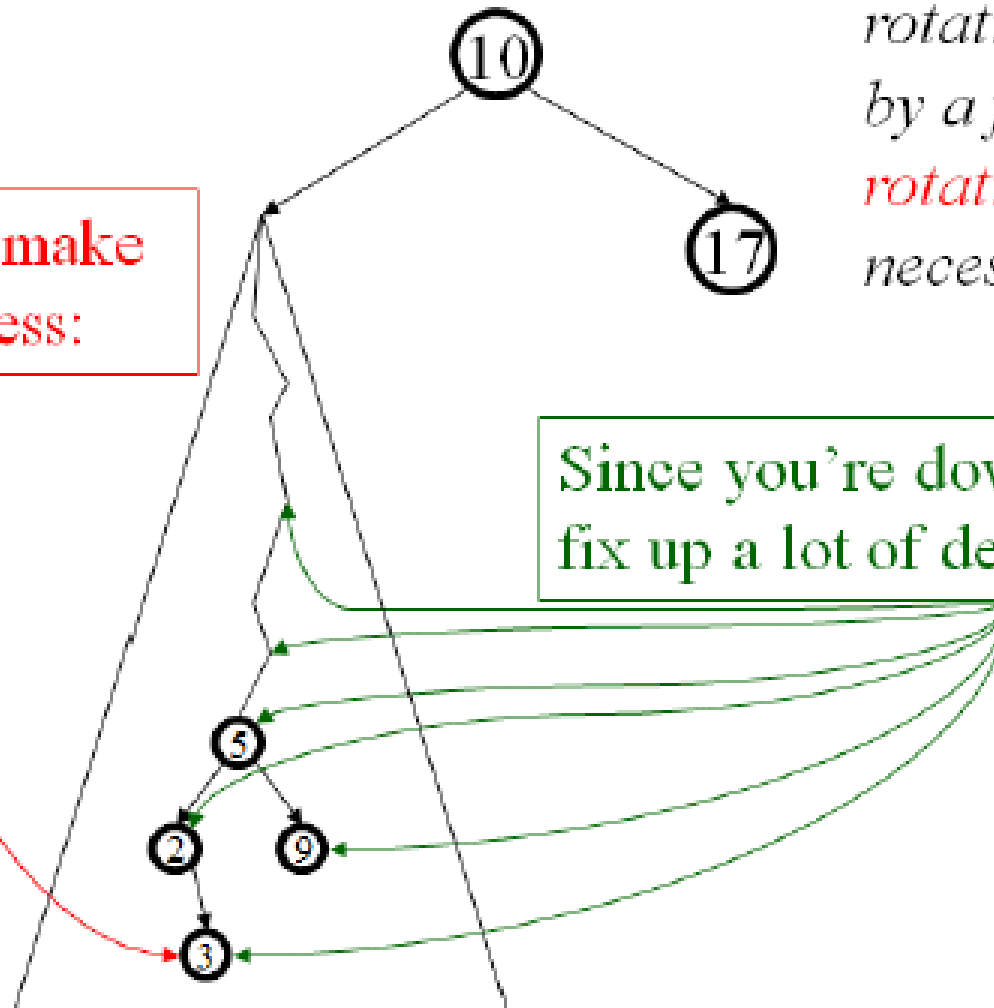
- Invented by Sleator and Tarjan (1985)
- Blind rebalancing – no height info kept!
- Worst-case time per operation is $O(n)$
- Worst-case amortized time is $O(\log n)$
- Insert/find always rotates node to the root!
- Good locality:
 - Most commonly accessed keys move high in tree – become easier and easier to find
 - Incorporate Move-to-Top strategy

IDEA

*move n to root by
series of **zig-zag**
and **zig-zig**
rotations, followed
by a final **single**
rotation (zig) if
necessary*

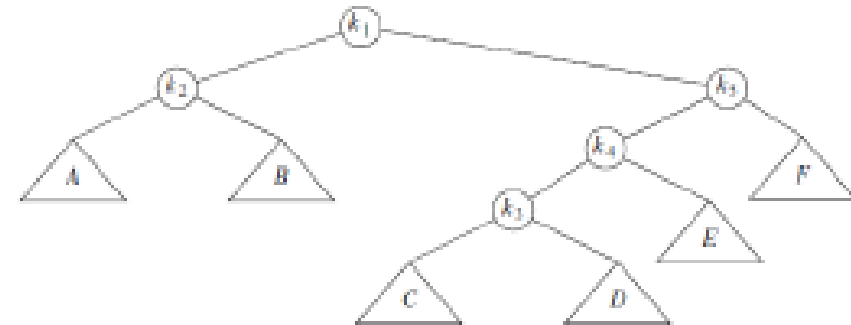
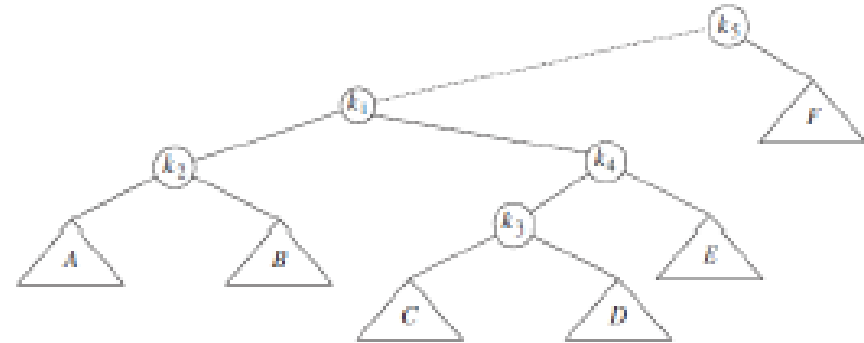
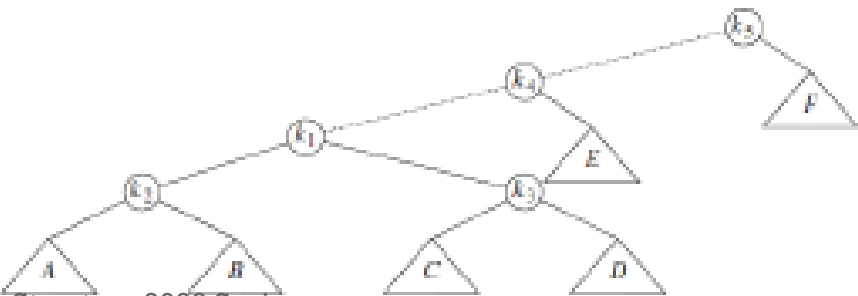
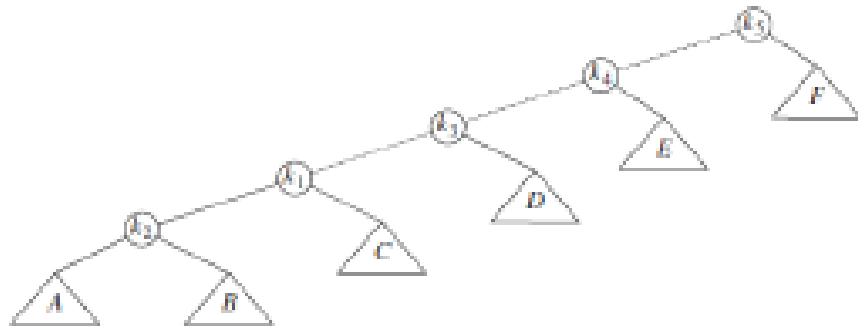
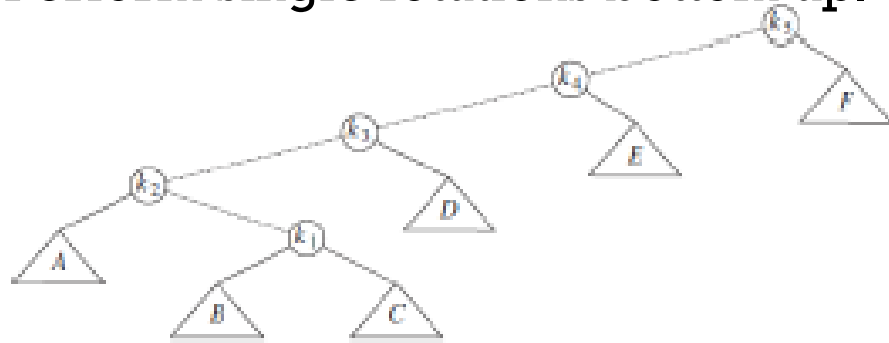
You're forced to make
a really deep access:

Since you're down there anyway,
fix up a lot of deep nodes!



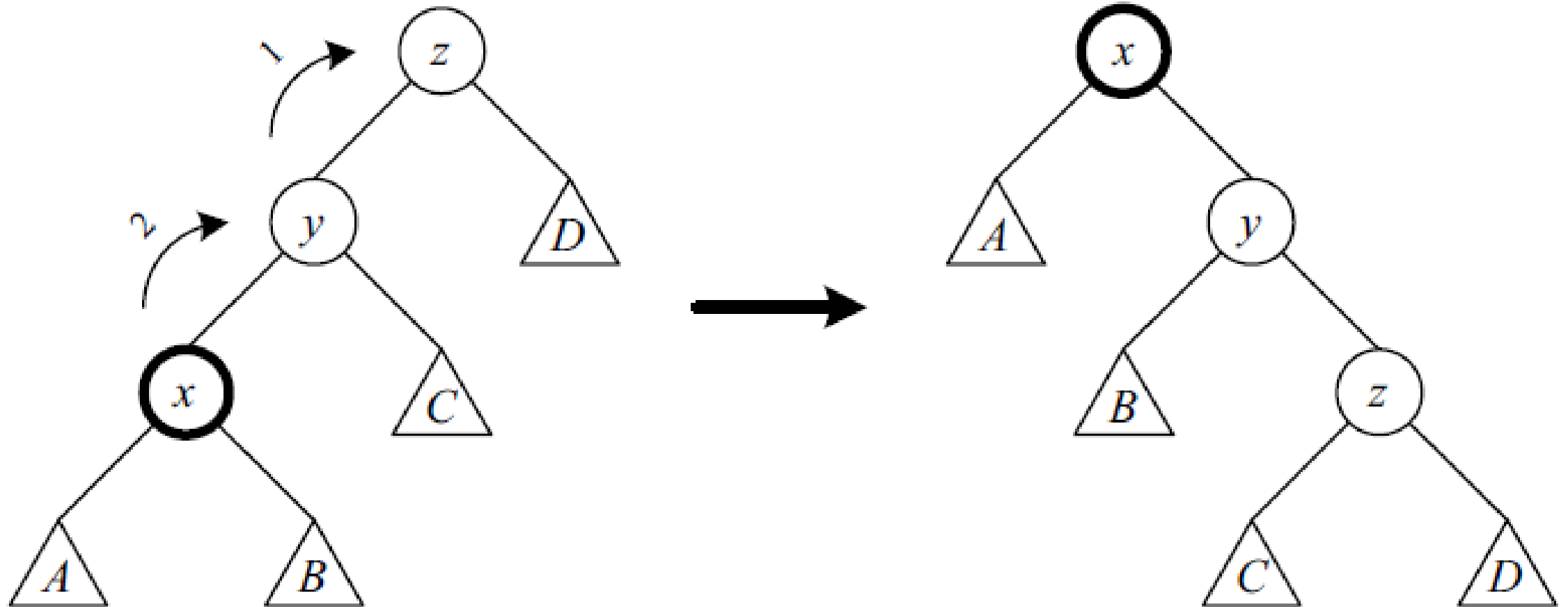
A SIMPLE IDEA (THAT DOES NOT WORK)

- Perform single rotations bottom up.

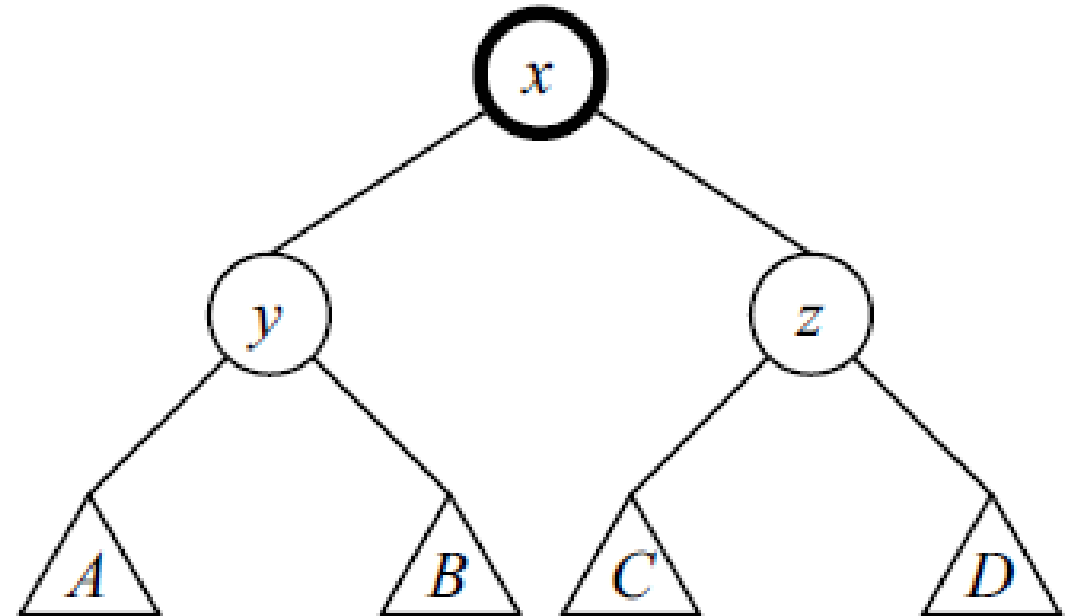
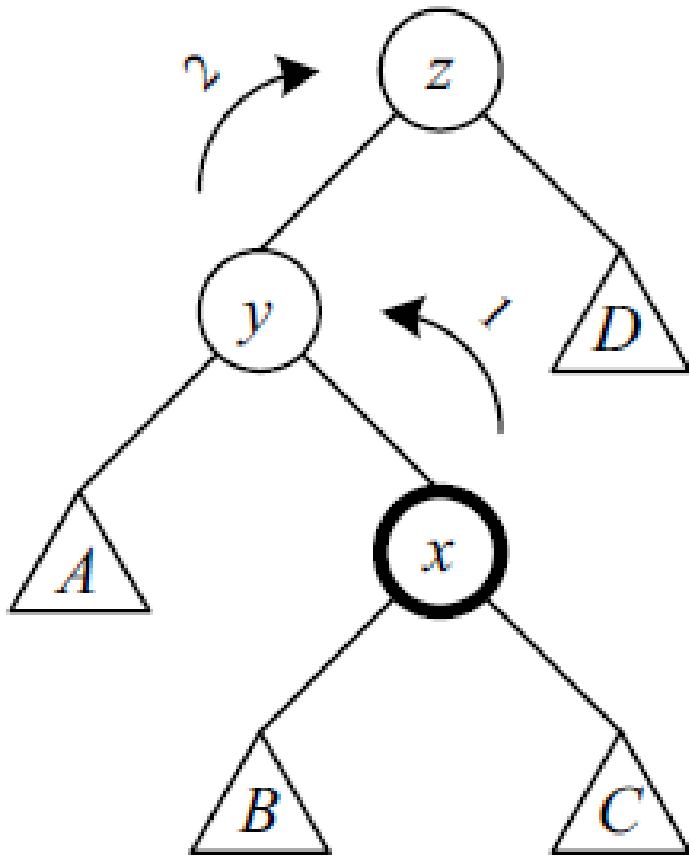


There is a sequence of M operations requiring $\Omega(MN)$ time, so this idea is not quite good enough.
(Homework problem!)

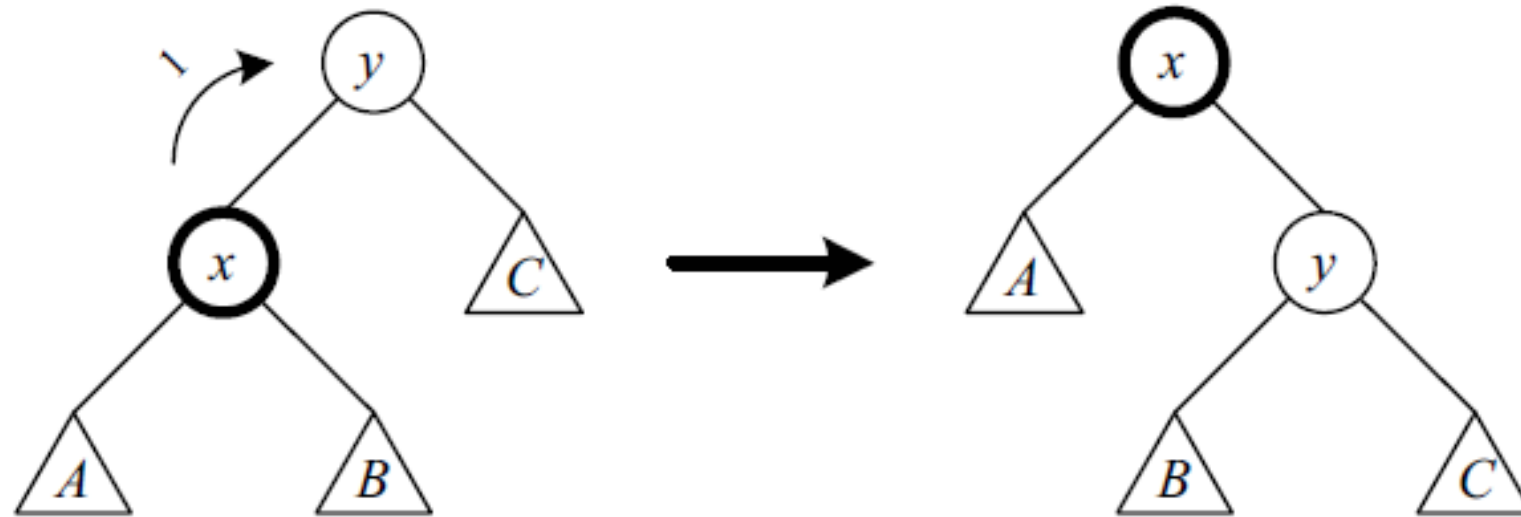
ZIG-ZIG



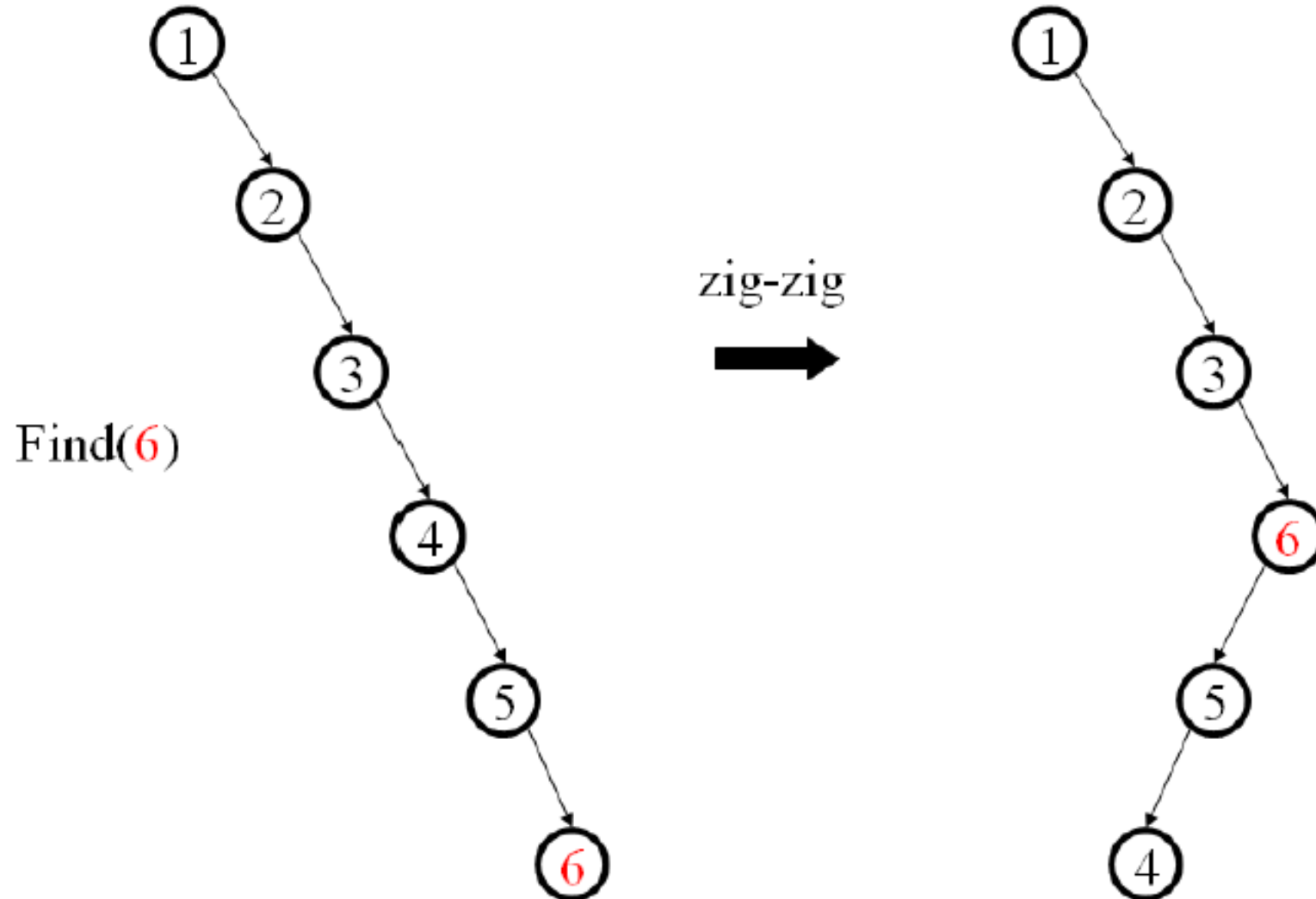
ZIG-ZAG



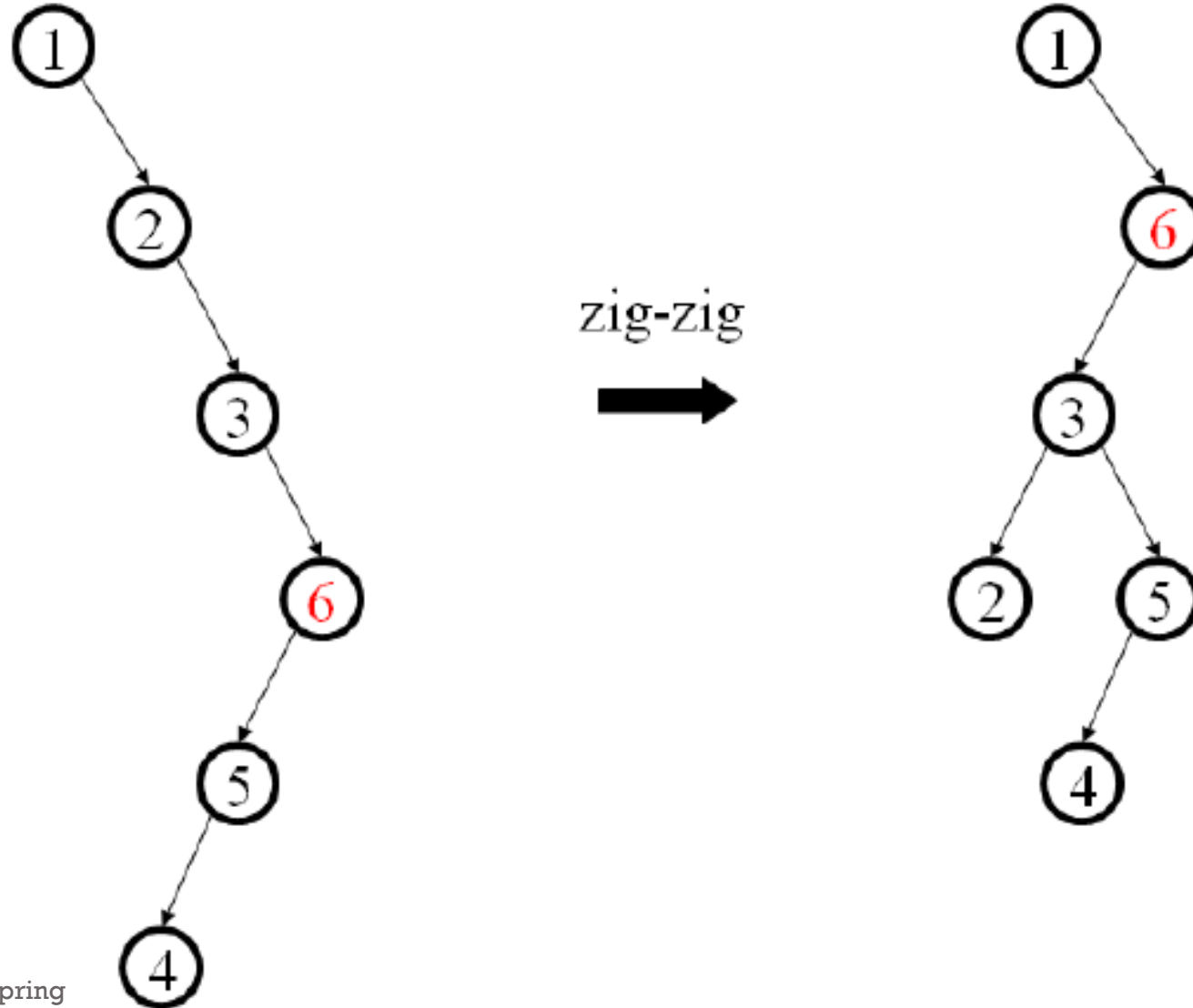
SIMPLE ROTATION



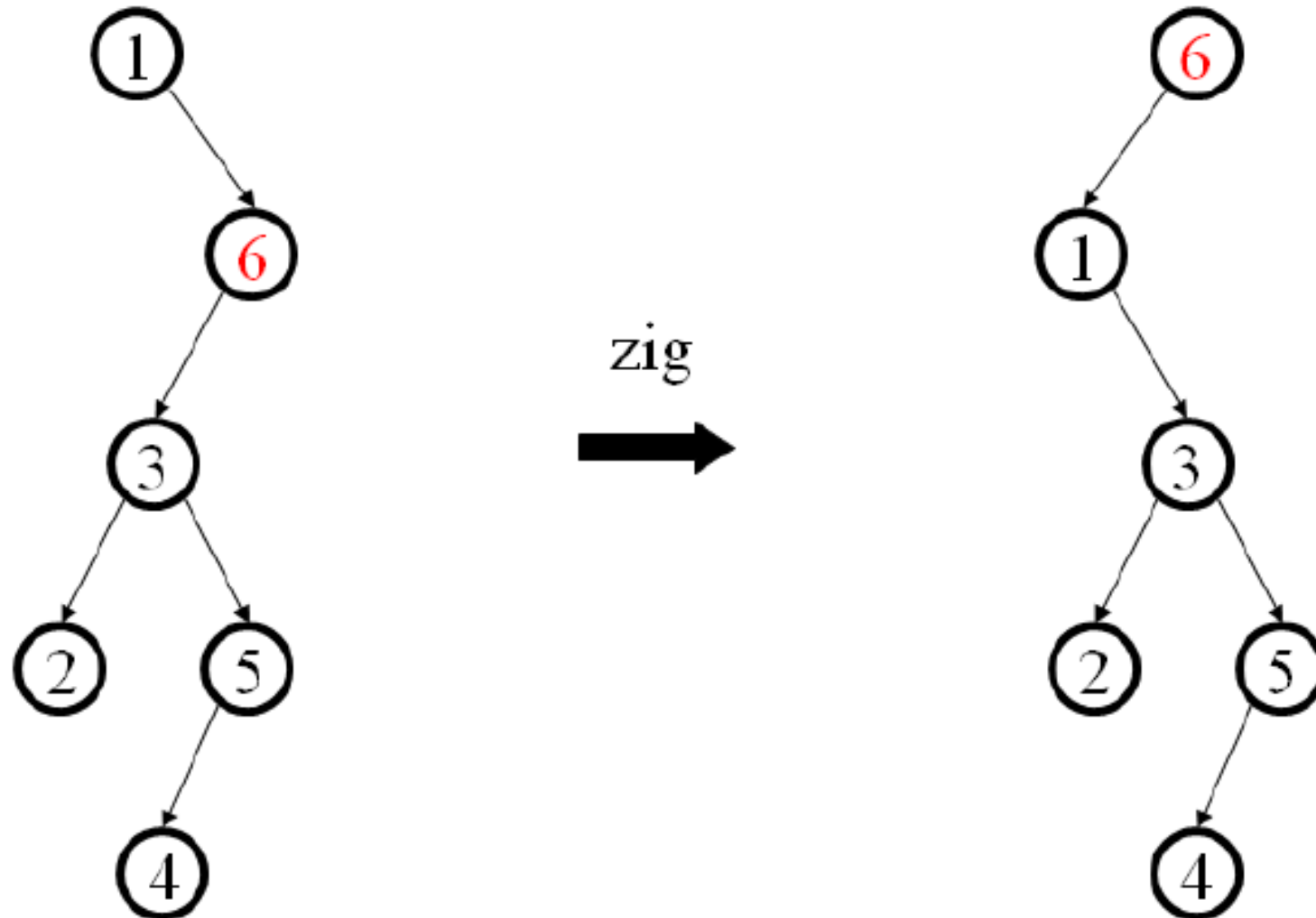
SPLAYING EXAMPLE



SPLAYING EXAMPLE (CONT'D)

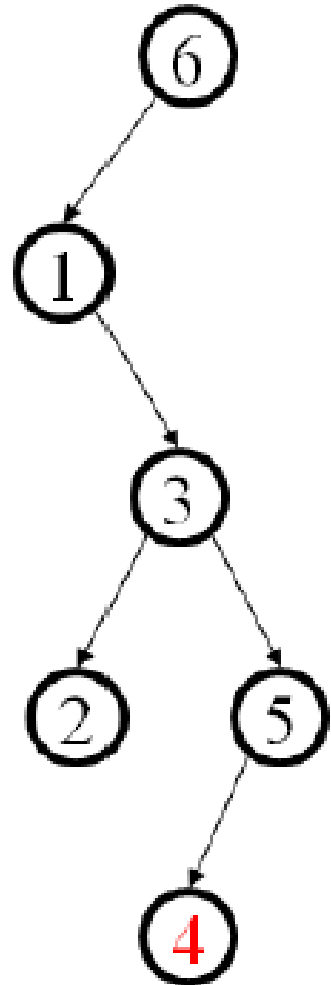


SPLAYING EXAMPLE (CONT'D)

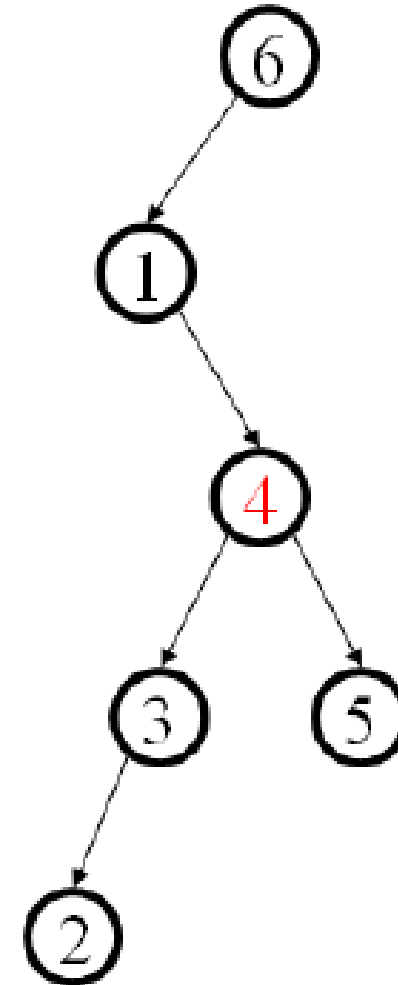


SPLAYING EXAMPLE (CONT'D)

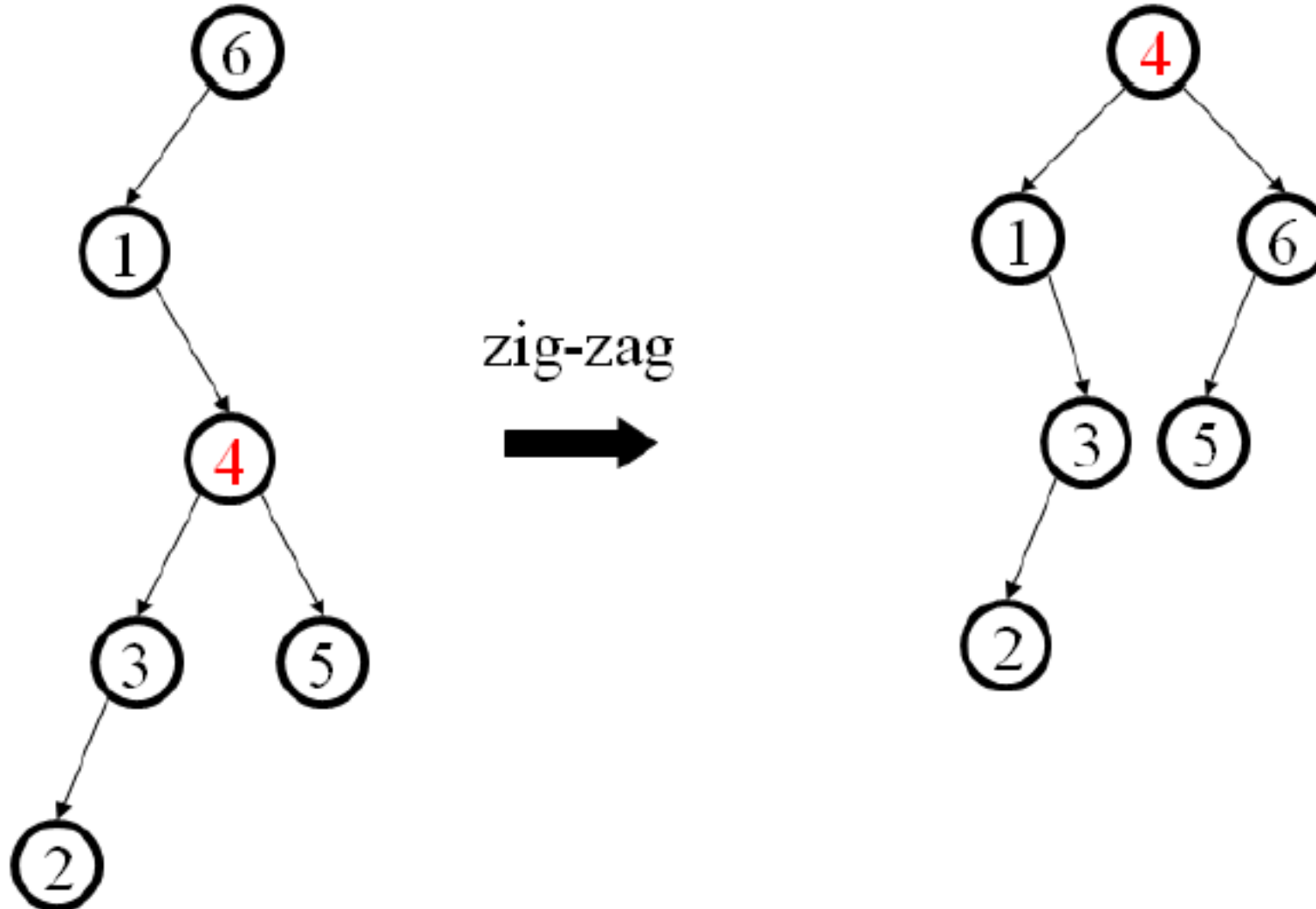
Find(4)



zig-zag
→

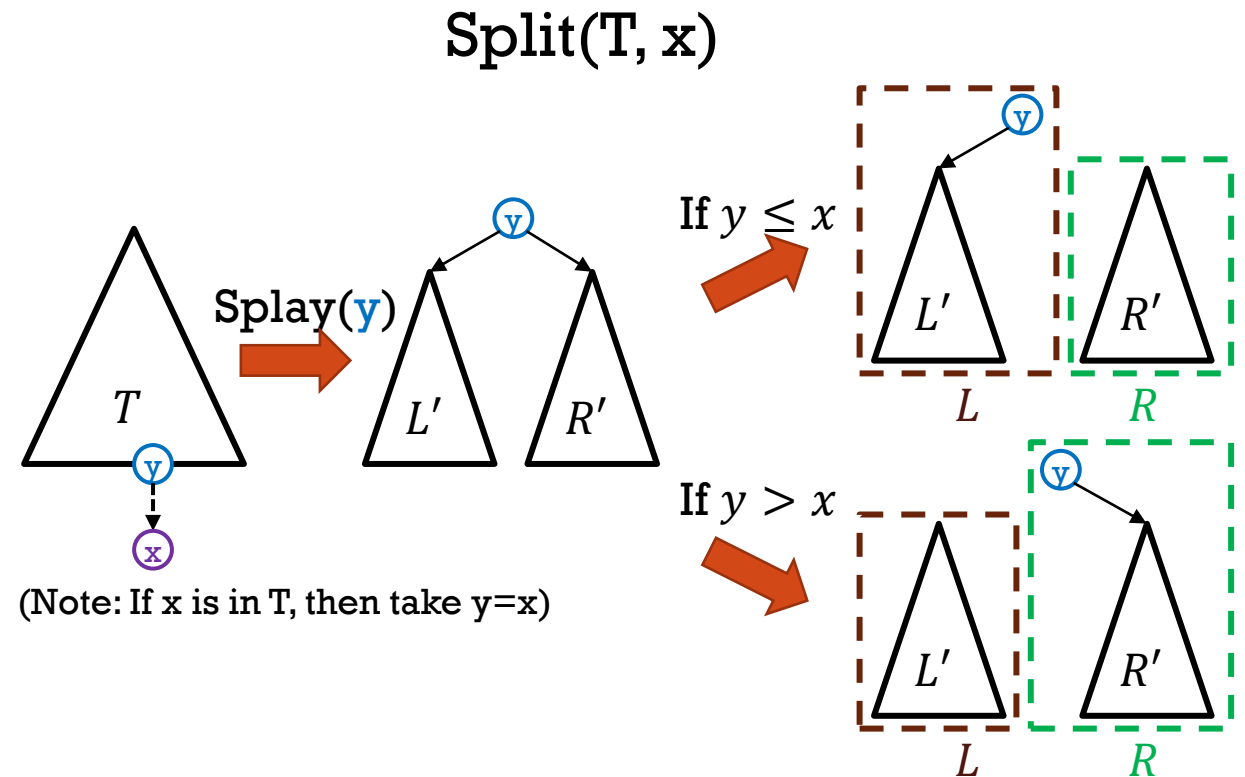


SPLAYING EXAMPLE (CONT'D)



SPLAY OPERATIONS: SPLIT

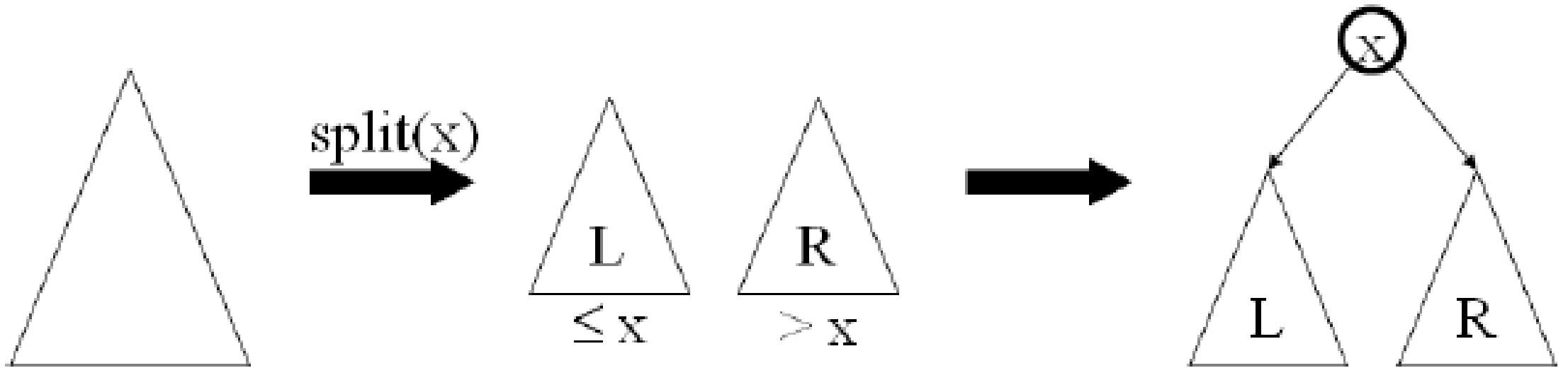
- To insert, could do an ordinary BST insert
 - but would not fix up tree
 - A BST insert followed by a find (splay)?
- Better idea: do the splay before the insert!
- How?
 - Split(T, x) creates two BSTs L and R
 - Insert as root, with children L and R



All elements of T are in either L or R

- All elements in L are $\leq x$
- All elements in R are $> x$
- L and R share no elements

SPLAY OPERATIONS: INSERT



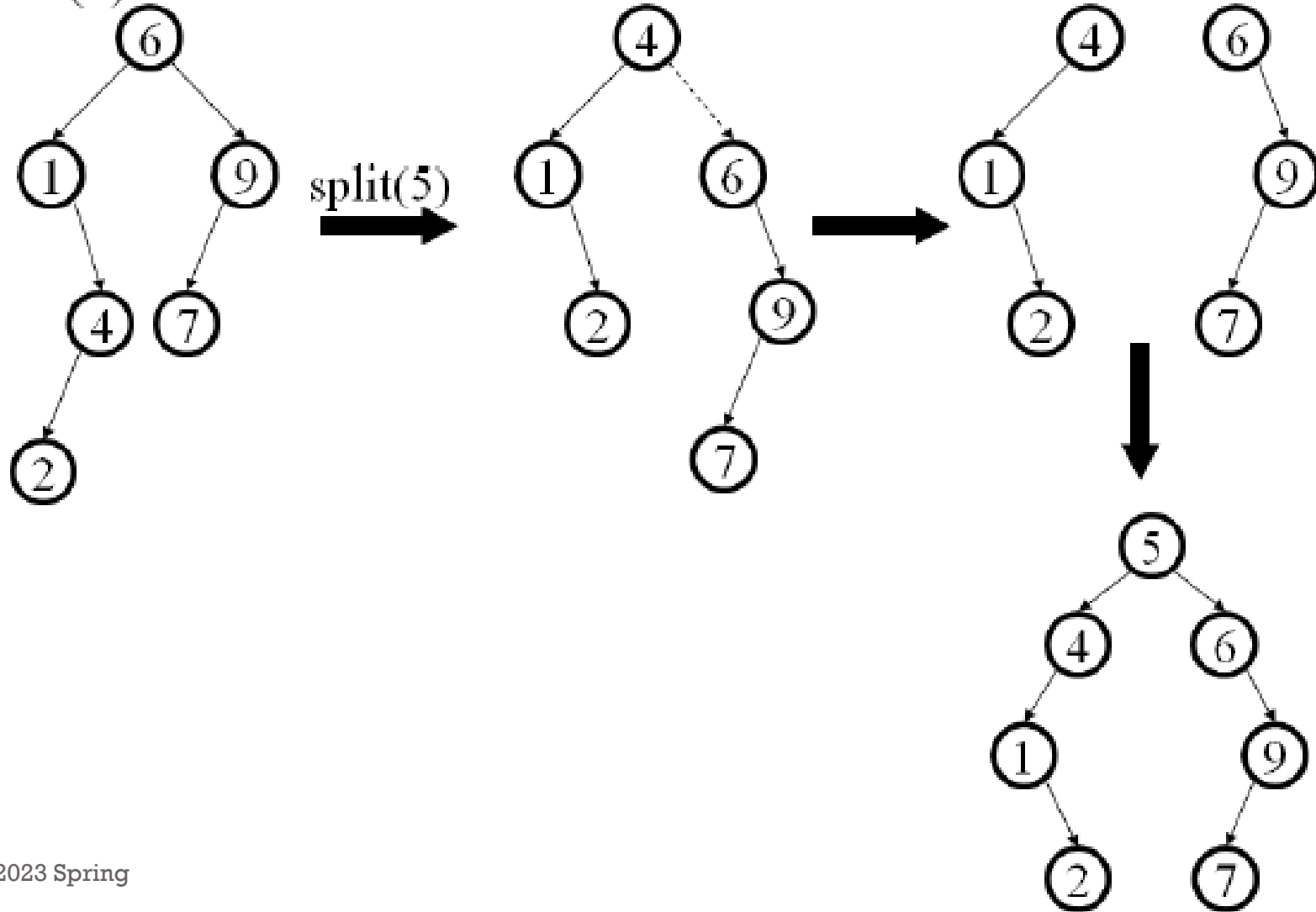
Insert(x) :

Split on x

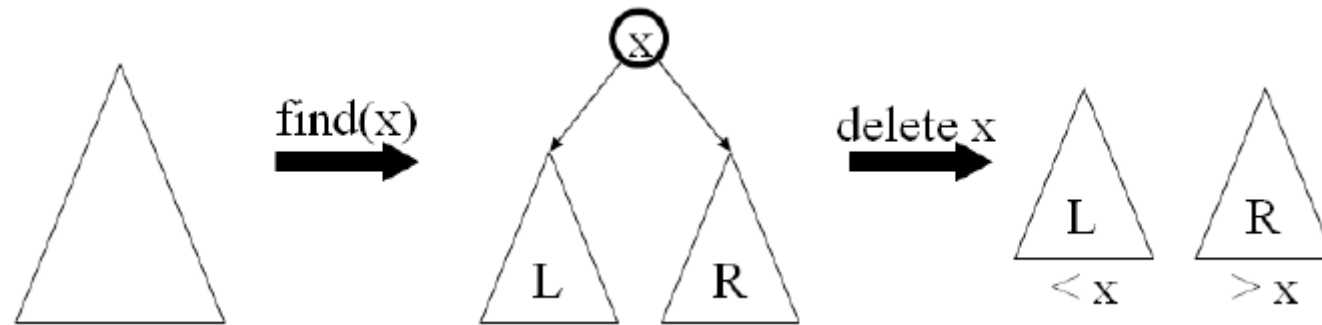
Join subtrees using x as root

INSERT EXAMPLE

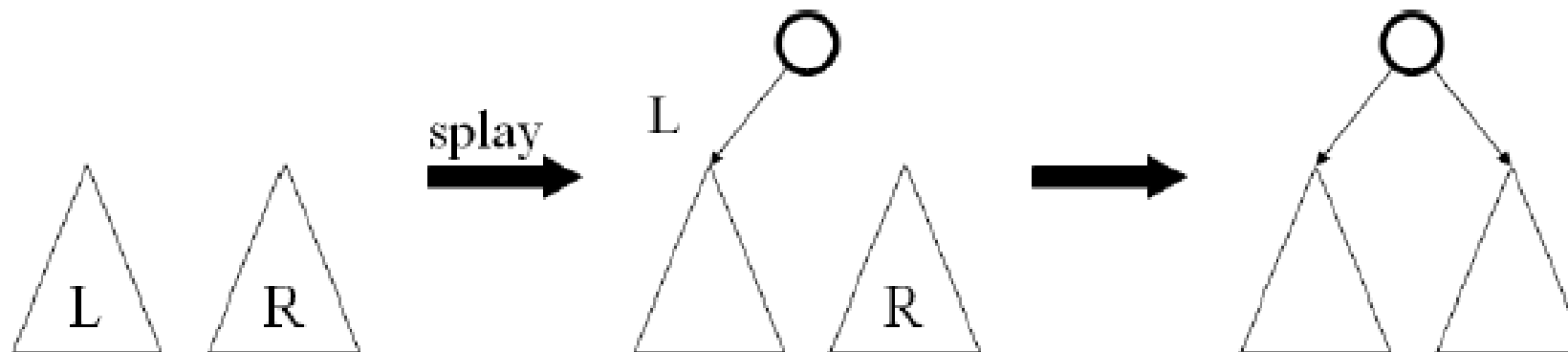
Insert(5)



DELETE

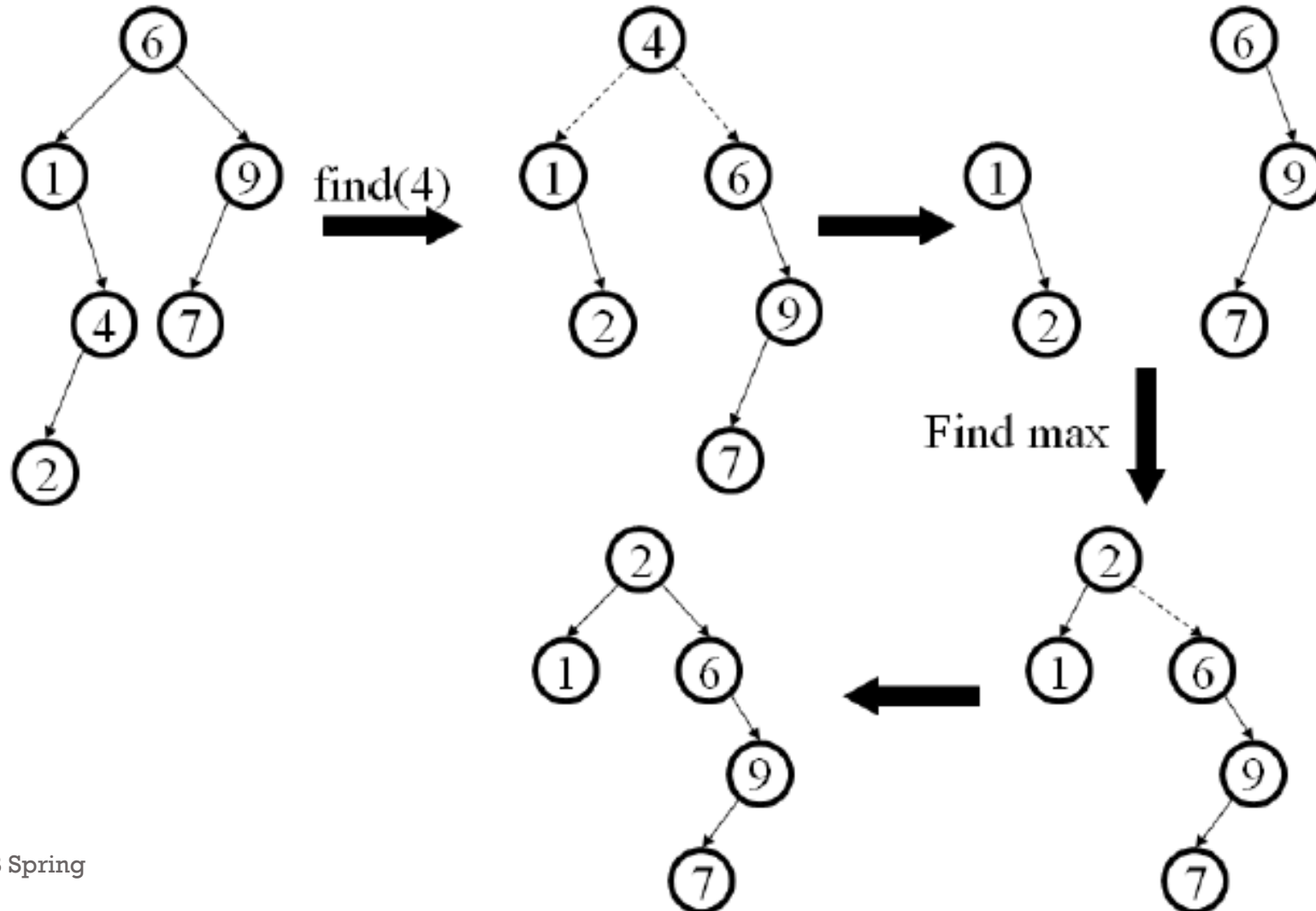


- $\text{Join}(L;R)$: given two trees such that $L < R$, merge them
 - Splay on the maximum element in L then attach R



DELETE EXAMPLE

Delete(4)



BOTTOM-UP SPLAY TREES ANALYSIS

- Actual complexity of each splay tree operation is the same as that of the associated splay.
- We can show that the amortized complexity of the splay operation is $O(\log n)$. (In the next few slides)
- Hence amortized complexity of search, insert, delete, and split are all $O(\log n)$.

AMORTIZED ANALYSIS

- Motivation:
 - Though a single operation may be very costly, in general most operations are okay.
 - Looking at the worst-case run time of one operation can be too pessimistic. We are interested in worst-case run time of a sequence of operations.
- Let P be a self-defined potential function, which measures how “messy” the current data structure is.
- Suppose at time step i , the data structure changes from $\mathcal{D}_{i-1}$ to $\mathcal{D}_i$ while the actual cost is $C(i)$, we say the amortized cost at time step i as

$$\text{amortized cost } A(i) = \text{potential change } P(\mathcal{D}_i) - P(\mathcal{D}_{i-1}) + \text{actual cost } C(i)$$

- Summing everything up from up to time step T yields

$$\sum_{i=1}^T \text{Total amortized cost } A(i) = \text{Total potential change } P(\mathcal{D}_T) - P(\mathcal{D}_0) + \sum_{i=1}^T \text{Total actual cost } C(i)$$

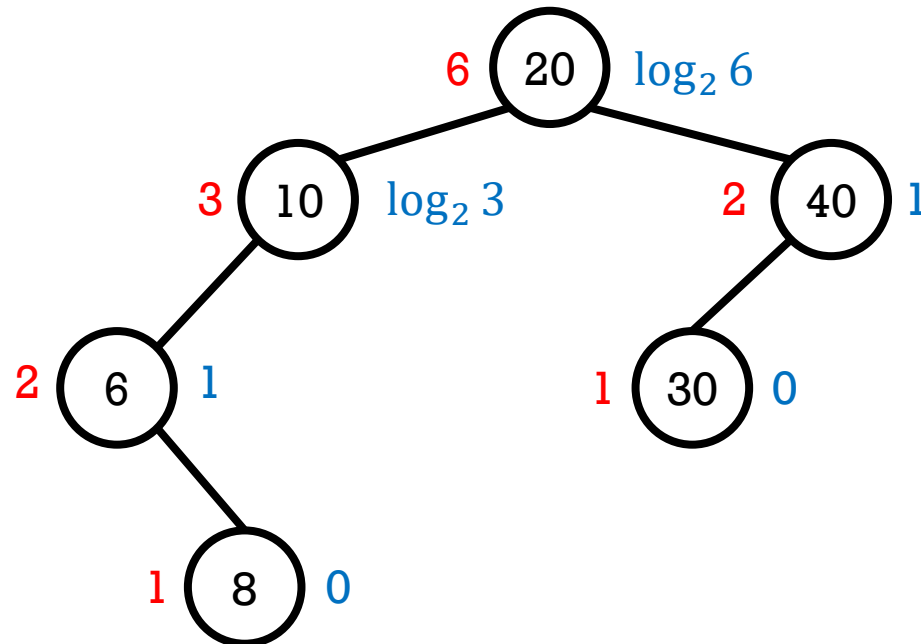
By proper choice of potential function P , amortized analysis may “smoothen out” the fluctuation of actual cost and provide better bounds over a sequence of operations.

$$\sum_{i=1}^T C(i) = P(\mathcal{D}_0) - P(\mathcal{D}_T) + \sum_{i=1}^T A(i)$$

POTENTIAL FUNCTION

- $\text{size}(x) = \text{\#nodes in subtree whose root is } x$.
- $\text{rank}(x) = \log_2 \text{size}(x)$.
- $P(i) = \sum_{\text{tree node } x} \text{rank}(x)$
 - $P(i)$ is potential after i 'th operation.
 - $\text{size}(x)$ and $\text{rank}(x)$ are computed after i 'th operation.
 - $P(0) = 0$.
- When join and split operations are done, number of splay trees > 1 at times.
 - $P(i)$ is obtained by summing over all nodes in all trees.

EXAMPLE



- $\text{size}(x)$ is in red.
- $\text{rank}(x)$ is in blue.
- $\text{Potential} = 2 + \log_2 3 + \log_2 6$.

SPLAY STEP AMORTIZED COST

■ Zig-zig

➤ Actual cost: 2 (rotations)

➤ The potential change is

$$\begin{aligned}\Delta P &= (r'(x) + r'(y) + r'(z)) - (r(x) + r(y) + r(z)) \\ &= (r'(x) - r(z)) + (r'(y) + r'(z)) - (r(x) + r(y)) \\ &\leq 0 + r'(x) + r'(z) - 2r(x)\end{aligned}$$

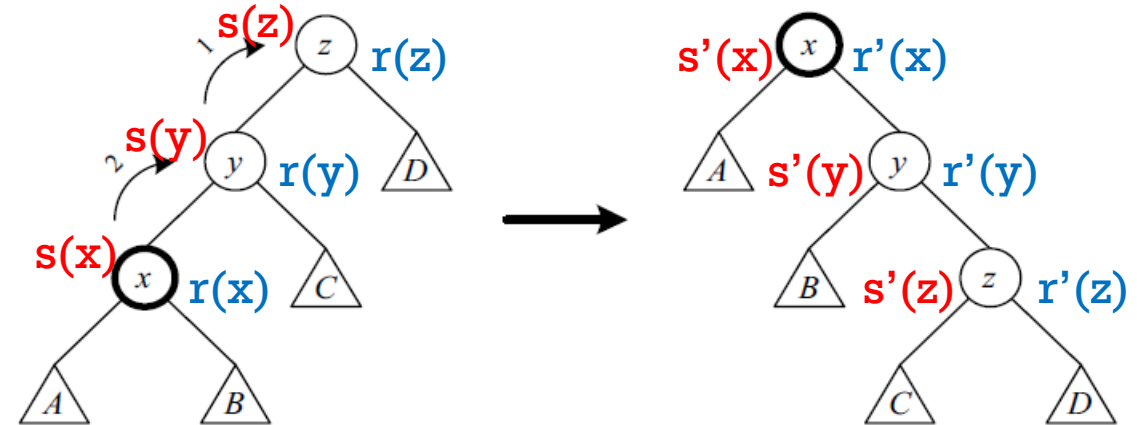
✓ The log function is concave, i.e., $\frac{\log_2 a + \log_2 b}{2} \leq \log_2 \left(\frac{a+b}{2} \right)$. Hence

$$\frac{r(x) + r'(z)}{2} = \frac{\log_2 s(x) + \log_2 s'(z)}{2} \leq \log_2 \left(\frac{s(x) + s'(z)}{2} \right) \leq \log_2 \left(\frac{s'(x)}{2} \right) = r'(x) - 1$$

✓ Therefore $r'(z) \leq 2r'(x) - r(x) - 2$

Hence $\Delta P \leq 3(r'(x) - r(x)) - 2$

➤ Amortized cost = Actual cost + $\Delta P \leq 3(r'(x) - r(x))$



SPLAY STEP AMORTIZED COST (CONT'D)

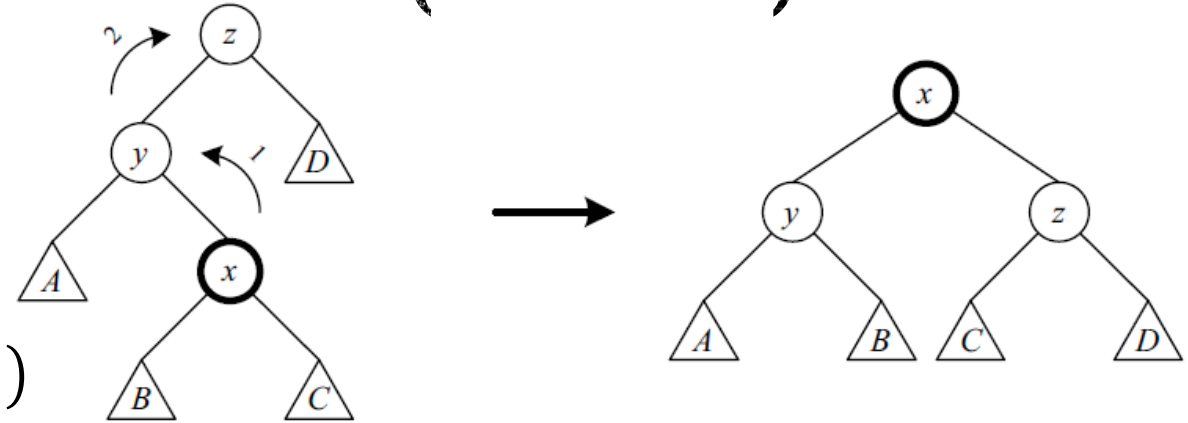
- Zig-zag

- Actual cost: 2 (rotations)

- The potential change is

$$\begin{aligned}\Delta P &= (r'(x) + r'(y) + r'(z)) - (r(x) + r(y) + r(z)) \\ &= (r'(x) - r(z)) + (r'(y) + r'(z)) - (r(x) + r(y)) \\ &\leq 0 + 2(r'(x) - 1) - 2r(x) \\ &= 2(r'(x) - r(x)) - 2\end{aligned}$$

- Amortized cost = Actual cost + $\Delta P \leq 2(r'(x) - r(x)) \leq 3(r'(x) - r(x))$



SPLAY STEP AMORTIZED COST (CONT'D)

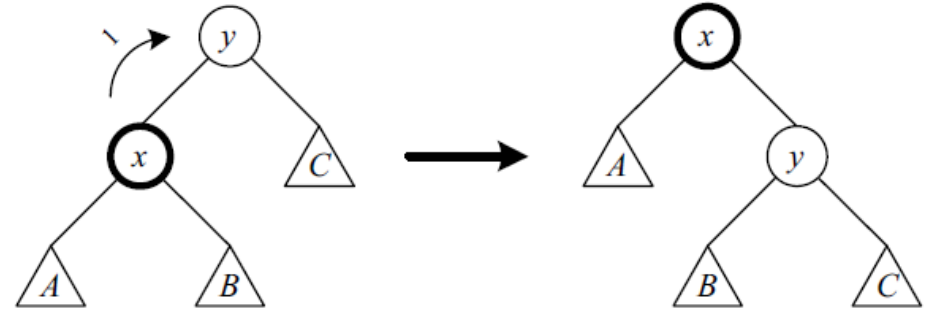
- Simple rotation

- Actual cost: 1 (rotation)

- The potential change is

$$\begin{aligned}\Delta P &= (r'(x) + r'(y)) - (r(x) + r(y)) \\ &= (r'(x) - r(y)) + (r'(y) - r(x)) \\ &\leq r'(x) - r(x)\end{aligned}$$

- Amortized cost = Actual cost + $\Delta P \leq (r'(x) - r(x)) + 1$



SPLAY STEP AMORTIZED COST (CONT'D)

- When x is not the root, zero or more 2-level splay steps followed by zero or one 1-level splay step.
- Let $r''(x)$ be rank of x just after last 2-level splay step.
- Let $r'''(x)$ be rank of x just after 1-level splay step
- Amortized cost of all 2-level splay steps $\leq 3(r''(x) - r(x))$
- Amortized cost of splay operation
$$\begin{aligned} &\leq 1 + r'''(x) - r''(x) + 3(r''(x) - r(x)) \\ &\leq 1 + 3(r'''(x) - r''(x)) + 3(r''(x) - r(x)) \\ &= 1 + 3(r'''(x) - r(x)) \\ &\leq 1 + 3(\log_2 n - r(x)) \in O(\log n) \end{aligned}$$

WEIGHTED ANALYSIS

- The above analysis can be generalized in the following way:
 - Assign to each node x a weight $w(x) > 0$
 - $\text{size}(x)$ = Sum of weights of descendants of node x .
 - $\text{rank}(x) = \log_2 \text{size}(x)$.
 - $P(\mathcal{D}_i) = \sum_{x \text{ in } \mathcal{D}_i} \text{rank}(x)$
- Note that the previous analysis corresponds to the special case $w(x) = 1$.
- The same analysis applies and the amortized cost of a splaying operation is again upper bounded by

$$1 + 3(\text{rank}(\text{root}) - \text{rank}(x)) = O\left(\log_2 \frac{W}{w(x)} + 1\right)$$

where W is the sum of all weights.

- The total cost is therefore bounded by

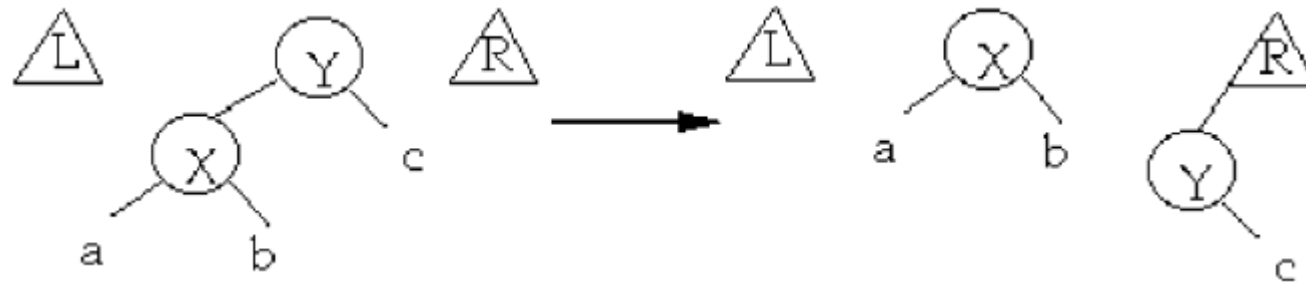
$$\sum_{i=1}^T C(i) = P(\mathcal{D}_0) - P(\mathcal{D}_T) + \sum_{i=1}^T A(i)$$

LOCALITY

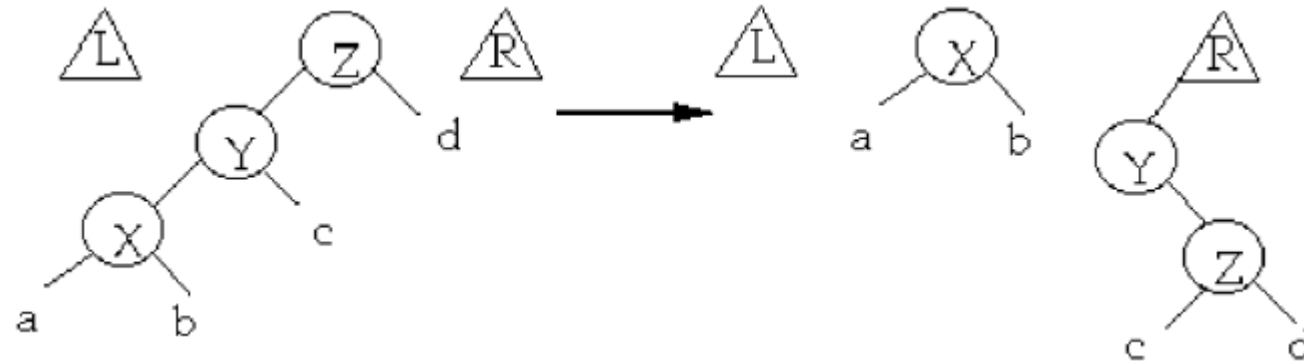
- Locality – if an item is accessed, it is likely to be accessed again soon.
- Assume $m \geq n$ access in a tree of size n .
 - Total worst case time is $O(m \log n)$
 - $O(\log n)$ per access amortized time
- Suppose only k distinct items are accessed in the m accesses.
 - Time is $O(n \log n + m \log k)$
 - ✓ Take $w(x) = \frac{1}{(1+t(x))^2}$, where $t(x) = \#$ distinct elements other than x accessed since the previous time x was accessed (See later slides for detailed proof)
 - ✓ Intuitive explanation:
 - $O(n \log n)$ – getting those k items near root
 - $m \log k$ – those k items are all at the top of the tree
 - Compare with $O(m \log n)$ for AVL tree

TOP-DOWN SPLAY

- **Case 1:** X is the node we are splaying

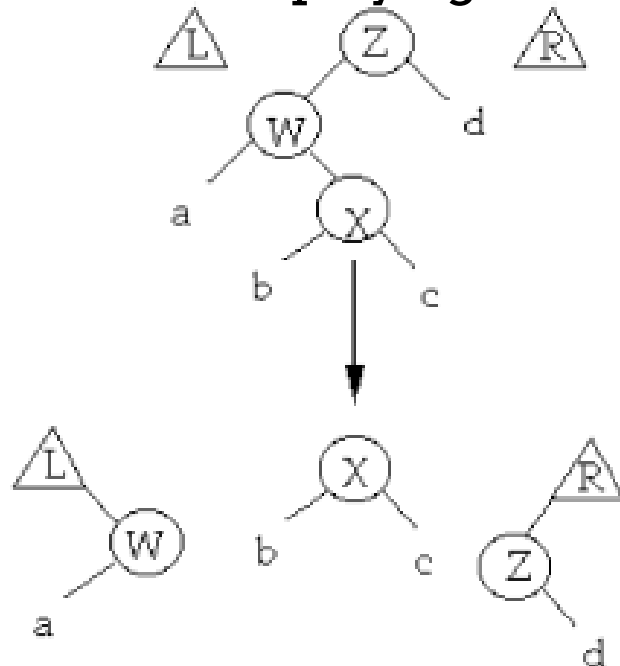


- **Case 2: (zig-zig)** The node we are splaying is in the subtree rooted at X

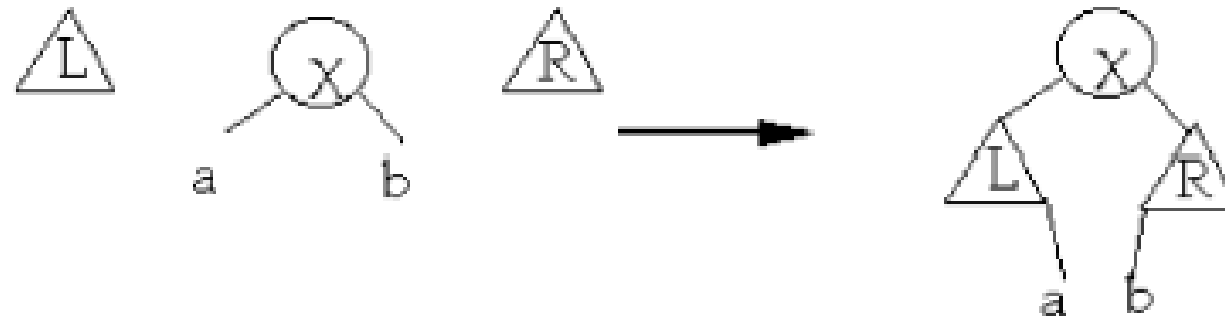


TOP-DOWN SPLAY (CONT'D)

- **Case 3: (zig-zag)** The node we are splaying is in the subtree rooted at X



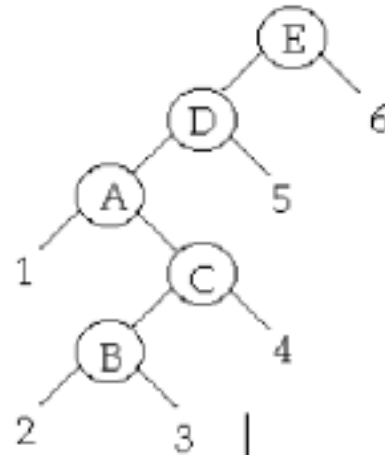
- **Case 4: (the last step)** X is the node we wish to splay



EXAMPLE SPLAY AT B

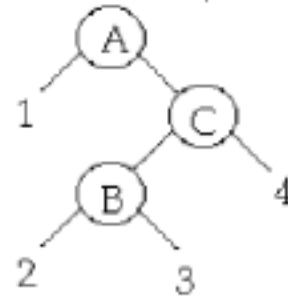
$\triangle L = \emptyset$

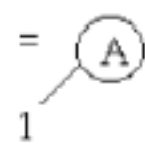
$\triangle R = \emptyset$



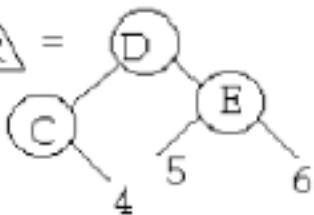
$\triangle L = \emptyset$

$\triangle R =$ 

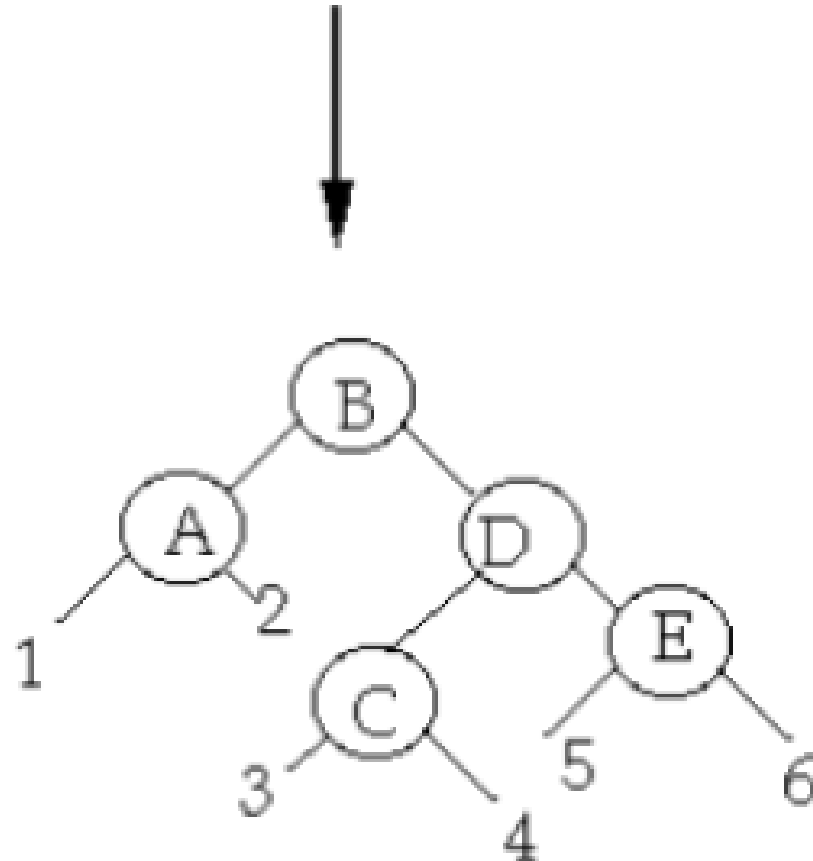
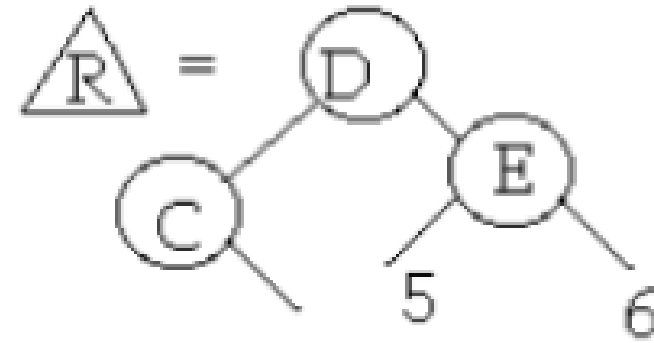
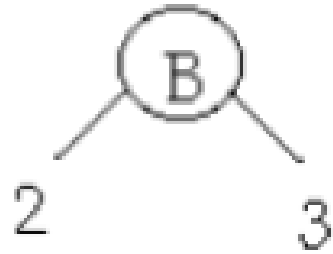
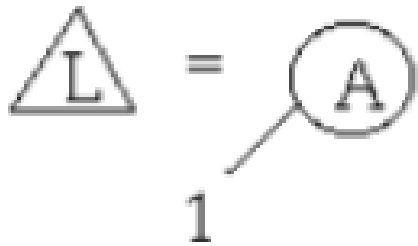


$\triangle L =$ 



$\triangle R =$ 

EXAMPLE SPLAY AT B (CONT'D)



DETAIL PROOF ON LOCALITY PROPERTY

- *We have mentioned that splay tree can exploit the locality property, namely*

the total actual cost of m accesses in a splay tree of size n , where only k distinct items are accessed, is $O(n \log n + m + m \log k)$.

Why??

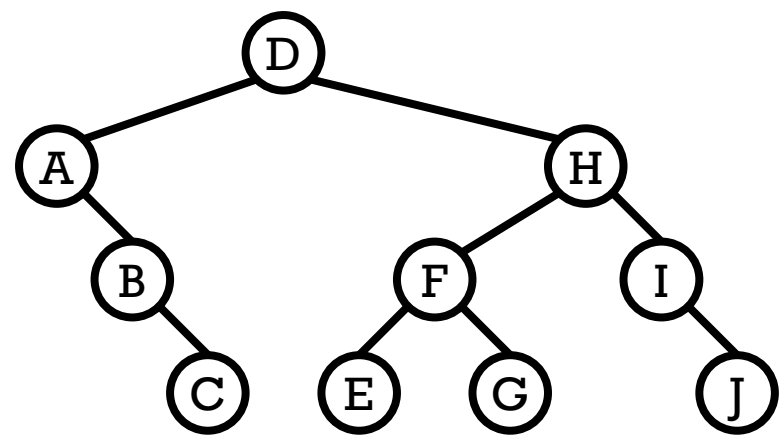
- *To answer this question, we first prove the **Working Set Theorem**:*

The total cost of m accesses in a splay tree of size n is

$$O\left(n \log n + m + \sum_{i=1}^m \log(1 + t_i(x_i))\right)$$

where the definition of $t_i(x_i)$ will be explained later (in the proof)

- Given a splay tree of size n:



- And a sequence of m accesses:
B、B、E、J、D、F、C、H、C、J、D、B
- Assign weights $1, \frac{1}{4}, \frac{1}{9}, \dots, \frac{1}{n^2}$ to the items in order by first access.
 - The item accessed earliest gets the largest weight; any **items never accessed** get the smallest weights.

item	A	B	C	D	E	F	G	H	I	J
weight	1/64	1/1	1/36	1/16	1/4	1/25	1/81	1/49	1/100	1/9

Alternative representation by order of weights:

weight	1/1	1/4	1/9	1/16	1/25	1/36	1/49	1/64	1/81	1/100
item	B	E	J	D	F	C	H	A	G	I

- During an access, suppose node **x** is accessed which has weight $1/k^2$.
- Do splay, and update weights after access:
 - Update the weight of node **x** as 1.
 - For each node **y** other than **x** which has weight $1/(k')^2$ with $k' < k$, updates the weight to $1/(k' + 1)^2$

weight	1	$\frac{1}{4}$	...	$\frac{1}{(k-1)^2}$	$\frac{1}{k^2}$	$\frac{1}{(k+1)^2}$	...	$\frac{1}{n^2}$
item	y_1	y_2	...	y_{k-1}	x	y_{k+1}	...	y_n

After weight updates:

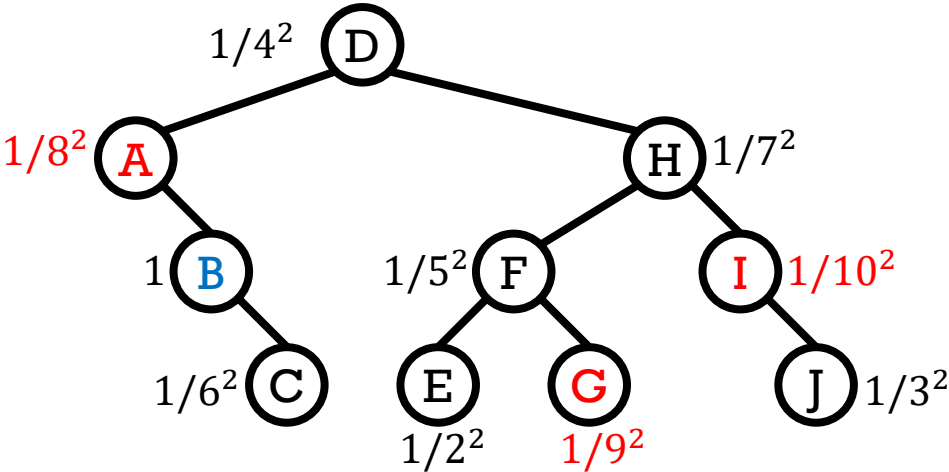
weight	1	$\frac{1}{4}$	...	$\frac{1}{(k-1)^2}$	$\frac{1}{k^2}$	$\frac{1}{(k+1)^2}$	...	$\frac{1}{n^2}$
item	x	y_1	...	y_{k-2}	y_{k-1}	y_{k+1}	...	y_n

- After weight update:
 - The weights are still a permutation of $1, \frac{1}{4}, \frac{1}{9}, \dots, \frac{1}{n^2}$.

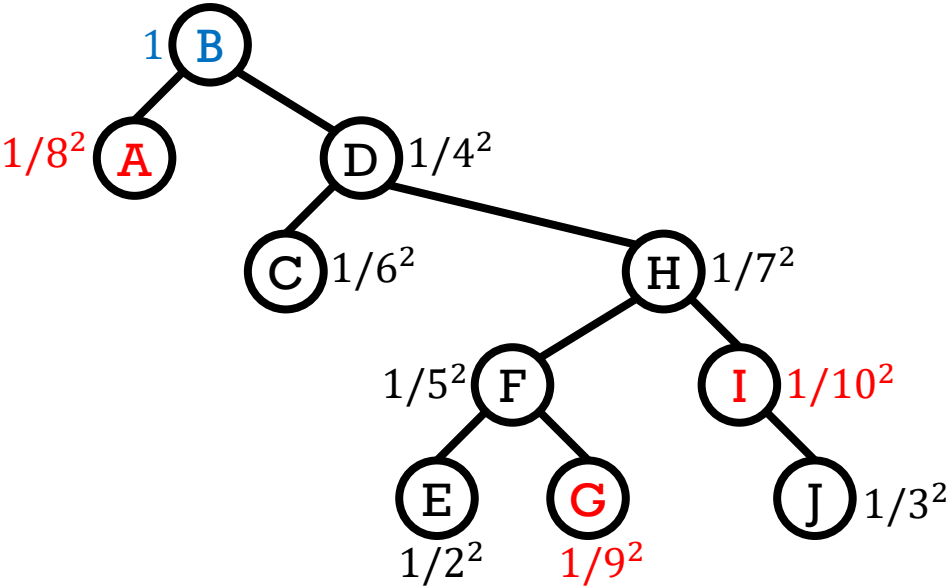
Proposition:

During each time step, If a item x has weight has weight $1/k^2$, then $k = 1 + \text{\# distinct items other than x accessed since the previous time x was accessed, unless x is never accessed in the whole m accesses.}$

Access **B**、B、E、J、D、F、C、H、C、J、D、B



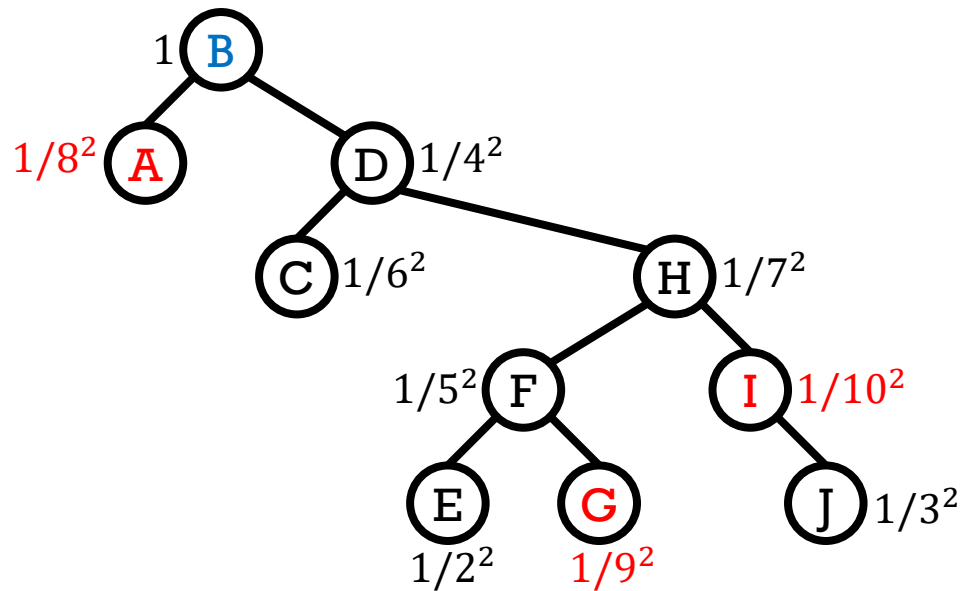
Splay(B)
➔



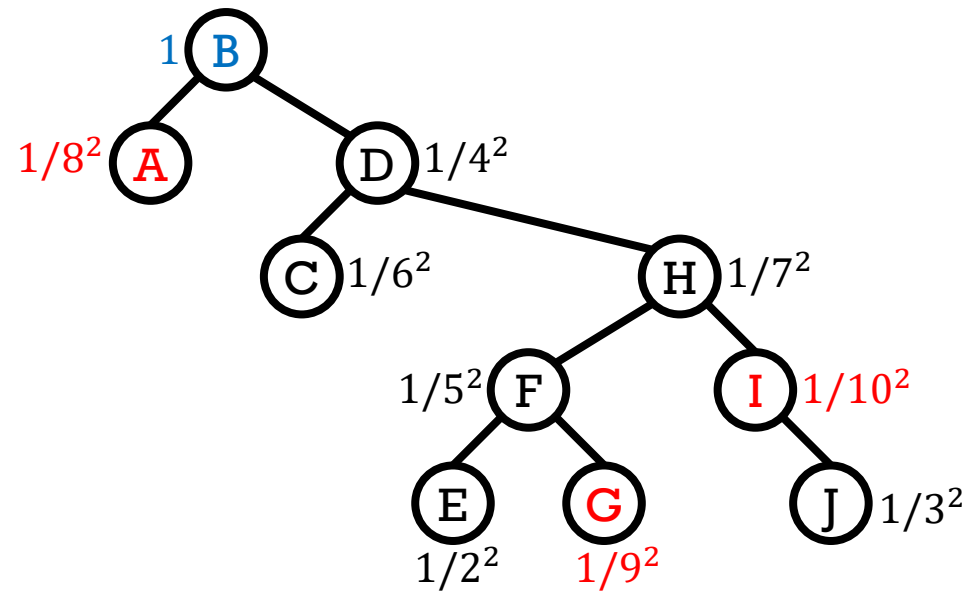
1/1	1/4	1/9	1/16	1/25	1/36	1/49	1/64	1/81	1/100
B	E	J	D	F	C	H	A	G	I

1/1	1/4	1/9	1/16	1/25	1/36	1/49	1/64	1/81	1/100
B	E	J	D	F	C	H	A	G	I

Access B、B、E、J、D、F、C、H、C、J、D、B



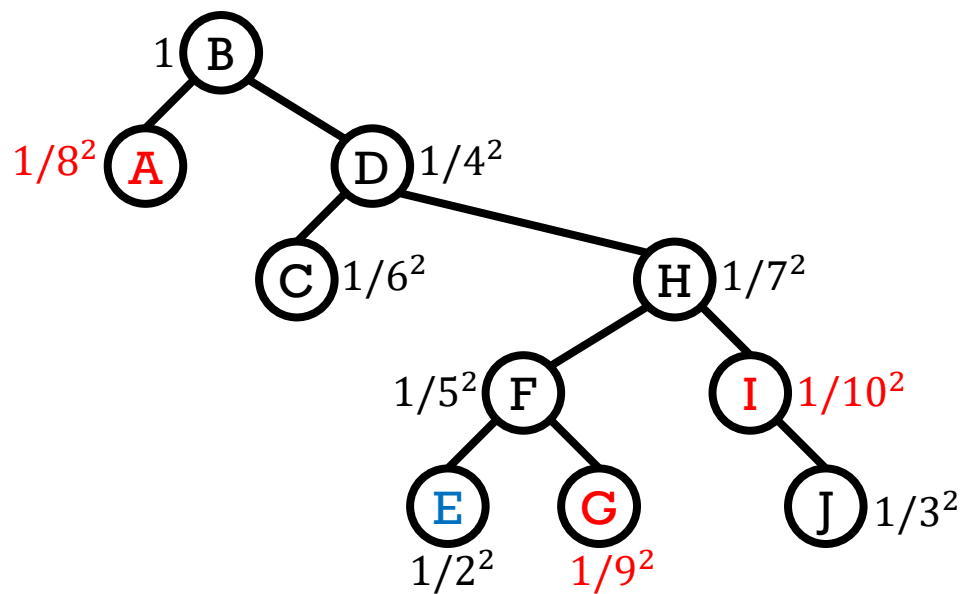
Splay(B)



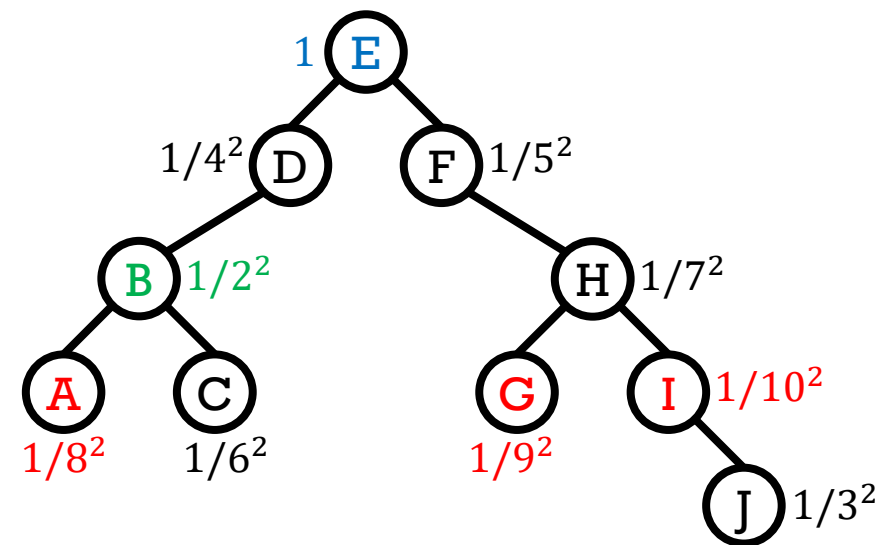
1/1	1/4	1/9	1/16	1/25	1/36	1/49	1/64	1/81	1/100
B	E	J	D	F	C	H	A	G	I

1/1	1/4	1/9	1/16	1/25	1/36	1/49	1/64	1/81	1/100
B	E	J	D	F	C	H	A	G	I

Access B、B、E、J、D、F、C、H、C、J、D、B



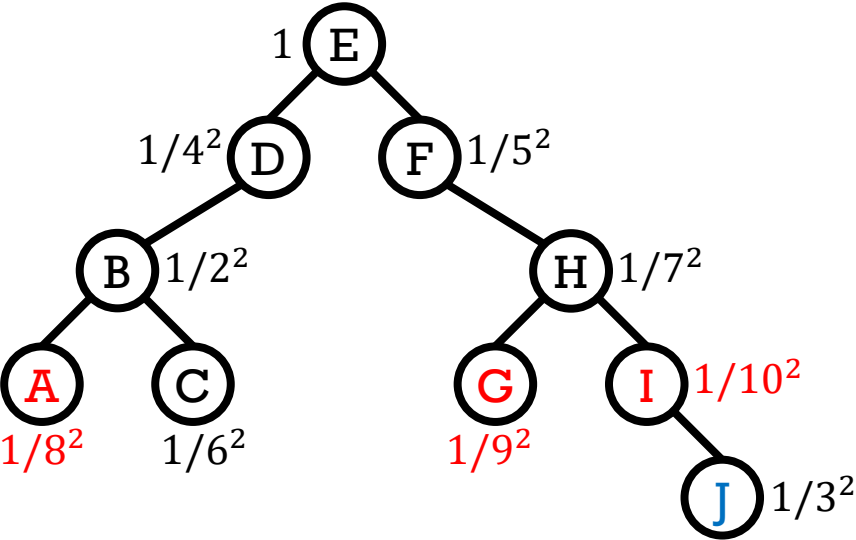
Splay(E)



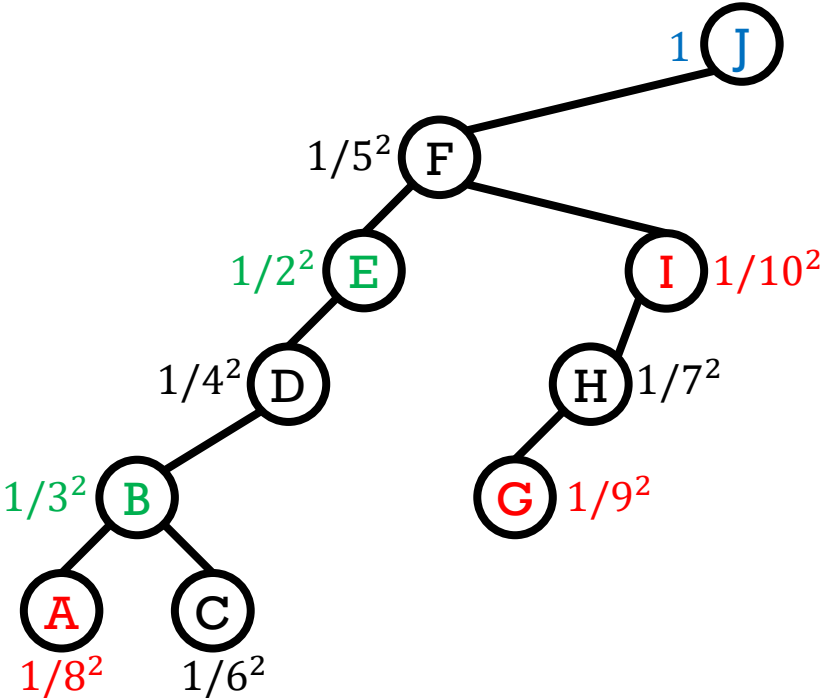
1/1	1/4	1/9	1/16	1/25	1/36	1/49	1/64	1/81	1/100
B	E	J	D	F	C	H	A	G	I

1/1	1/4	1/9	1/16	1/25	1/36	1/49	1/64	1/81	1/100
E	B	J	D	F	C	H	A	G	I

Access B、B、E、J、D、F、C、H、C、J、D、B



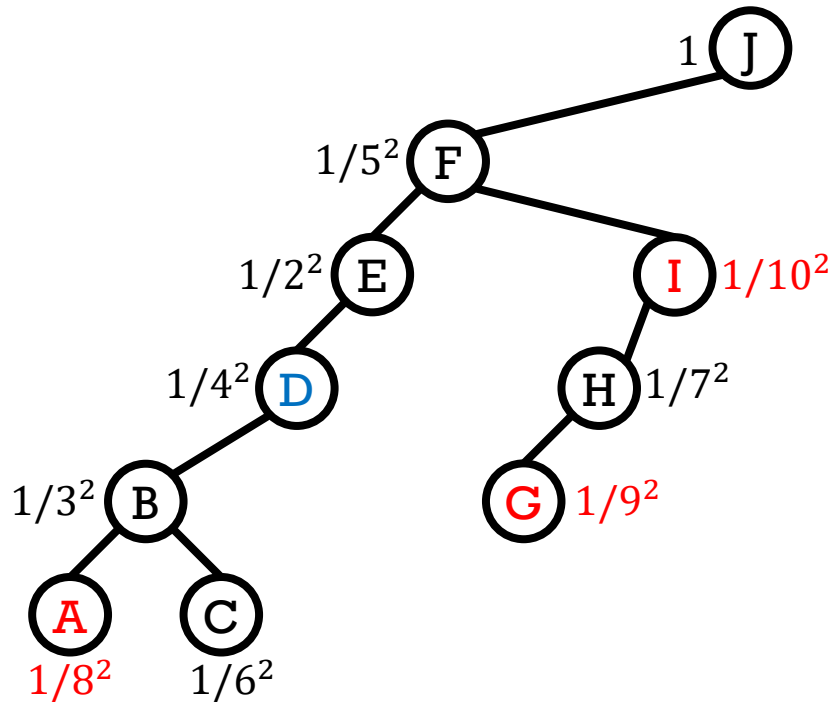
Splay(J)



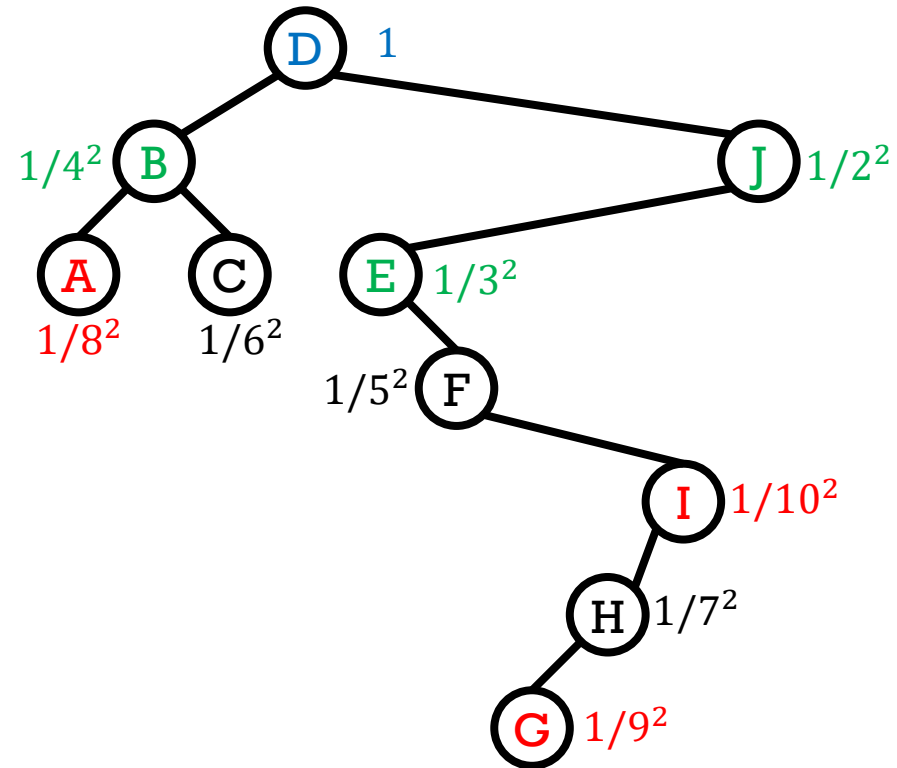
1/1	1/4	1/9	1/16	1/25	1/36	1/49	1/64	1/81	1/100
E	B	J	D	F	C	H	A	G	I

1/1	1/4	1/9	1/16	1/25	1/36	1/49	1/64	1/81	1/100
J	E	B	D	F	C	H	A	G	I

Access B、B、E、J、D、F、C、H、C、J、D、B



Splay(D)



1/1	1/4	1/9	1/16	1/25	1/36	1/49	1/64	1/81	1/100
J	E	B	D	F	C	H	A	G	I

1/1	1/4	1/9	1/16	1/25	1/36	1/49	1/64	1/81	1/100
D	J	E	B	F	C	H	A	G	I

WORKING SET THEOREM PROOF

- For the i -th access, splay operation

➤ Amortized cost is at most

$$1 + 3(\text{rank}(\text{root}) - \text{rank}(x_i)) \leq 1 + 3 \log_2 \frac{W}{w_i(x_i)}$$

where W is the sum of all weights, and x_i is the item accessed.

✓ Note that the weights may change with time, so we use subscript i to highlight its dependency to step i .

✓ Note that the weights are always a permutation of $1, \frac{1}{4}, \frac{1}{9}, \dots, \frac{1}{n^2}$.

➔ W does not depend on i , and that $W \leq 1 + \frac{1}{4} + \dots + \frac{1}{n^2} < \frac{\pi^2}{6}$.

✓ Express $w_i(x) = \frac{1}{(1+t_i(x))^2}$. Note that $t_i(x)$ = # distinct items other than x

accessed since the previous time x was accessed (snapshot at the beginning of the i -th access), **unless x is never accessed in the whole m accesses.**

➤ Amortized cost is at most $O(1 + \log(1 + t_i(x_i)))$.

WORKING SET THEOREM PROOF

- For the i -th access, weight update:
 - Only increases the weight of the item in the root (because splaying at the accessed item moves it to the root) and only decreases the weights of other items in the tree.
 - The size of the root is unchanged, but the sizes of other nodes can decrease. → Only decreases the potential.
 - We do not maintain the weights (it's only for analysis purpose → No actual cost).
 - Amortized cost is at most 0.

To sum up, the amortized cost for the i -th access is upper bounded by $O(1 + \log(1 + t_i(x)))$

$$\sum_{i=1}^m C(i) = P(\mathcal{D}_0) - P(\mathcal{D}_m) + \sum_{i=1}^m \boxed{A(i)} \\ O(1 + \log(1 + t_i(x_i)))$$

WORKING SET THEOREM PROOF

- Bounding the potential function:

- Note that $\frac{1}{n^2} \leq w_i(x) \leq 1$, hence

$$\frac{1}{n^2} \leq size_i(x) \leq W \Rightarrow -2 \log_2 n \leq rank_i(x) \leq \log_2 \frac{\pi^2}{6}$$

- Hence potential ranges from $-2n \log_2 n$ to $n \log_2 \frac{\pi^2}{6}$.

- Hence potential drop is $O(n \log n)$.

To sum up, the actual time for m accesses in a splay tree of size n is bounded by

$$\sum_{i=1}^m C(i) = \underbrace{P(\mathcal{D}_0) - P(\mathcal{D}_m)}_{O(n \log n)} + \sum_{i=1}^m \underbrace{A(i)}_{O(1 + \log(1 + t_i(x_i)))}$$

Working Set Theorem

LOCALITY PROOF

- Recall by working set theorem that

$$O\left(n \log n + \sum_{i=1}^m (1 + \log(1 + t_i(x_i)))\right) = O\left(n \log n + m + \sum_{i=1}^m \log(1 + t_i(x_i))\right)$$

Want to further simplify this,
which can be written as

$$\sum_{x \in \text{Tree}} \sum_{i: x_i = x} \log(1 + t_i(x))$$

各自取 log 之後再加起來

Also Observe that

$$\sum_{i: x_i = x} (1 + t_i(x)) \leq m$$

B、**B**、**E**、**J**、**D**、**F**、**C**、**H**、**C**、**J**、**D**、**B**

$$\begin{aligned} \sum_{i: x_i = B} \log(1 + t_i(B)) &\leq \text{---} \text{---} \text{---} \\ \sum_{i: x_i = E} \log(1 + t_i(E)) &\leq \text{---} \\ \sum_{i: x_i = J} \log(1 + t_i(J)) &\leq \text{---} \text{---} \\ \sum_{i: x_i = D} \log(1 + t_i(D)) &\leq \text{---} \text{---} \\ \sum_{i: x_i = F} \log(1 + t_i(F)) &\leq \text{---} \\ \sum_{i: x_i = C} \log(1 + t_i(C)) &\leq \text{---} \text{---} \\ \sum_{i: x_i = H} \log(1 + t_i(H)) &\leq \text{---} \end{aligned}$$

LOCALITY PROOF

- By inequality of arithmetic and geometric means (算幾不等式)

$$\prod_{i:x_i=x} (1 + t_i(x)) \leq \left(\frac{1}{m_x} \sum_{i:x_i=x} (1 + t_i(x)) \right)^{m_x} \leq \left(\frac{m}{m_x} \right)^{m_x}$$

where $m_x = \#$ times x is accessed. Hence

$$\sum_{i:x_i=x} \log(1 + t_i(x)) \leq m_x \log \left(\frac{m}{m_x} \right)$$

- Denote $p_x = \frac{m_x}{m}$, then

$$\sum_{x \in \text{Tree}} \sum_{i:x_i=x} \log(1 + t_i(x)) \leq m \sum_{x \in \text{Tree}} p_x \log \left(\frac{1}{p_x} \right)$$

Entropy of the probability distribution ($p_x: x \in \text{Tree}$)

- If only k distinct items are accessed, then p_x is only nonzero for k items x .
- The maximum entropy of a probability distribution on k items is $\log k$ (the equiprobable case)

Hence, the total actual cost of m accesses in a splay tree of size n , where only k distinct items are accessed, is $O(n \log n + m + m \log k)$

SUMMARY

- Splay trees are arguably the most practical kind of self-balancing trees
- If number of finds is much larger than n , then locality is crucial!
- Also supports efficient Split and Join operations – useful for other tasks